



شماره دانشجویی:

۸۱۰۱۰۱۴۹۲



تئوری زنجیره‌های بلوکی
نام و نام خانوادگی: علیرضا کریمی

تمرین شماره ۴

سوال ۱

(a)

تحلیل جامع: چرا ۳ سرور برای تحمل ۱ سرور معیوب در مدل $PBFT$ کافی نیست؟
در یک سیستم توزیع شده، برای رسیدن به توافق در حضور سرورهای فریبکار (بیزانسی)، باید از قوانینی پیروی کرد که صحت و پایداری سیستم را تضمین کنند. الگوریتم $PBFT$ این قوانین را ارائه می‌دهد، اما این قوانین نیازمند حداقل تعداد مشخصی سرور هستند. در ادامه نشان می‌دهیم که چرا یک سیستم ۳ سروری ($n = 3$) نمی‌تواند حتی ۱ سرور معیوب ($f = 1$) را تحمل کند.

بخش اول: توضیح شهودی (مشکل از دیدگاه کلاینت)
بیاید ابتدا مشکل را از دیدگاه کلاینت (سرویس گیرنده) که منتظر پاسخ است، بررسی کنیم.

مثال و توضیح:

۱. شرایط سیستم:

• تعداد کل سرورها: $(n) = 3$ ($S1, S2, S3$)

• حداکثر سرورهای معیوب: $(f) = 1$

• سرور معیوب: فرض کنیم $S3$ سرور بیزانسی (خائن) است.

• سرورهای صادق: $S1$ و $S2$ صادق هستند.

۲. ارسال و پردازش درخواست:

• یک کلاینت درخواستی را ارسال می‌کند که نتیجه صحیح آن " A " است.

• $S1$ (صادق): درخواست را پردازش کرده، به نتیجه صحیح " A " می‌رسد و آن را برای کلاینت ارسال می‌کند.

• $S2$ (صادق): او نیز به نتیجه صحیح " A " می‌رسد، اما به دلیل تأخیر در شبکه، پاسخ او هنوز به دست کلاینت نرسیده است.

• $S3$ (خائن): به جای نتیجه درست، یک پاسخ فریبکارانه یعنی " B " را برای کلاینت ارسال می‌کند.

۳. وضعیت از دید کلاینت: در این لحظه، کلاینت دو پاسخ متناقض دریافت کرده است: "A" از $S1$ و "B" از $S3$. کلاینت در یک بن‌بست کامل قرار دارد، زیرا نمی‌تواند تشخیص دهد کدام سرور راست می‌گوید:

- احتمال اول: $S1$ صادق و $S3$ خائن است (پاسخ صحیح: A).

- احتمال دوم: $S3$ صادق و $S1$ خائن است (پاسخ صحیح: B).

چون کلاینت نمی‌تواند به رسیدن پاسخ از $S2$ مطمئن باشد (شاید $S2$ از کار افتاده باشد)، هیچ راهی برای رسیدن به یک اکثریت قاطع و انتخاب پاسخ درست ندارد. این سناریوی ساده نشان می‌دهد که سیستم از دید کاربر نهایی قابل اعتماد نیست.

بخش دوم: تحلیل فنی (شکست الگوریتم از درون)

حالا بیایید ببینیم چرا این بن‌بست بیرونی رخ می‌دهد. دلیل آن، شکست فرآیند اجماع در فازهای داخلی الگوریتم $PBFT$ است.

قانون طلایی $PBFT$: برای اینکه هر مرحله از اجماع ($Prepare$ و $Commit$) با موفقیت انجام شود، هر سرور باید $2f + 1$ پیام یکسان از دیگران جمع‌آوری کند. در مثال ما ($f = 1$)، این عدد برابر ۳ است. تحلیل گام به گام فازهای $PBFT$:

مرحله ۱: درخواست و پیشنهاد اولیه (فاز $Pre - Prepare$)

- شرایط: $S1$ رهبر ($Primary$)، $S2$ صادق و $S3$ خائن است.

- فرآیند: کلاینت درخواستی برای اجرای تراکنش A ارسال می‌کند. $S1$ به عنوان رهبر، یک پیام $PRE - PREPARE$ به $S2$ و $S3$ می‌فرستد و پیشنهاد اجرای این تراکنش را می‌دهد. تا اینجا، هر دو سرور صادق ($S1$ و $S2$) پیشنهاد را به درستی دریافت کرده‌اند.

مرحله ۲: بن‌بست در فاز تأیید (فاز $Prepare$) اینجا نقطه شکست سیستم است.

۱. اقدام سرورهای صادق: $S1$ و $S2$ پس از دریافت پیشنهاد، هر کدام یک پیام $PREPARE$ به کل شبکه می‌فرستند تا آمادگی خود را اعلام کنند.

۲. اقدام سرور خائن: سرور $S3$ برای مختل کردن سیستم، سکوت می‌کند و هیچ پیام $PREPARE$ ای ارسال نمی‌کند.

۳. شمارش آرا از دید سرورهای صادق:

- از دید $S1$: یک پیام $PREPARE$ از خودش و یک پیام از $S2$ دریافت می‌کند. مجموع: ۲ رأی.

- از دید $S2$: یک پیام $PREPARE$ از خودش و یک پیام از $S1$ دریافت می‌کند. مجموع: ۲ رأی.

نتیجه فاز $Prepare$: هر دو سرور صادق روی عدد ۲ متوقف شده‌اند، در حالی که برای عبور از این مرحله به ۳ رأی نیاز دارند. آن‌ها تا ابد منتظر رأی سوم از $S3$ می‌مانند که هرگز نمی‌رسد. سیستم در این فاز قفل می‌شود.

مرحله ۳: عدم امکان رسیدن به فاز نهایی (فاز $Commit$)

چون سرورها هرگز به حد نصاب آرا در فاز *Prepare* نرسیدند، هیچ‌گاه به حالت "آماده" (*prepared*) در نمی‌آیند و در نتیجه، هرگز پیام *COMMIT* ارسال نخواهند کرد. فرآیند اجماع به طور کامل متوقف می‌شود و تراکنش هرگز اجرا نمی‌شود.

نتیجه‌گیری نهایی بن‌بستی که کلاینت در بخش اول با آن مواجه شد (دریافت پاسخ‌های متناقض یا عدم دریافت پاسخ کافی)، یک علامت بیرونی از یک مشکل درونی است. مشکل اصلی این است که ساختار ریاضی و قوانین رأی‌گیری الگوریتم *PBFT* با ۳ سرور و ۱ خطای بیزانسی کار نمی‌کند. سرورهای صادق از نظر الگوریتمی توانایی جمع‌آوری آرای کافی $(2f + 1)$ برای نهایی کردن تصمیم را ندارند و این نقص، خود را به شکل یک سیستم غیرقابل اعتماد به کاربر نهایی نشان می‌دهد. به همین دلیل، برای تحمل f خطا، حداقل $3f + 1$ سرور یک الزام مطلق است.

(b)

اثبات جامع لزوم $n \geq 3f + 1$ برای تحمل خطای بیزانسی در یک سیستم توزیع‌شده، هدف رسیدن به یک توافق یکپارچه بر سر یک حقیقت واحد است. اما این فرآیند زمانی پیچیده می‌شود که برخی از نودها (سرورها) دچار خطای بیزانسی شوند. همانطور که در متون اشاره شد، خطای بیزانسی بدترین نوع خطاست، زیرا نود معیوب صرفاً از کار نمی‌افتد، بلکه ممکن است فعالانه تلاش کند تا سیستم را با ارسال اطلاعات متناقض به نودهای مختلف، فریب دهد. برای مقابله با این تهدید، یک سیستم باید از قوانینی پیروی کند تا حتی در حضور f نود خائن، همچنان به درستی کار کند. اثبات اینکه چرا حداقل تعداد نودها (n) باید $3f + 1$ باشد، از سه دیدگاه مختلف قابل بررسی است: دیدگاه شهودی کلاینت، دیدگاه ریاضی محض، و دیدگاه الگوریتمی *PBFT*.

بخش اول: اثبات شهودی (دیدگاه کلاینت و نیاز به اکثریت قاطع)

ابتدا از دیدگاه کلاینت (سرویس‌گیرنده) که منتظر یک پاسخ قابل اعتماد است، به موضوع نگاه می‌کنیم. هدف کلاینت: انتخاب پاسخ صحیح از میان n پاسخ دریافتی، با علم به اینکه حداکثر f پاسخ می‌تواند دروغ باشد. سوال اصلی: حداقل چند پاسخ یکسان (k) لازم است تا کلاینت مطمئن شود که پاسخ صحیح است؟ برای پاسخ، بدترین سناریو را در نظر می‌گیریم: فرض می‌کنیم تمام f سرور خائن، همگی یک پاسخ دروغین یکسان را ارسال کرده‌اند تا کلاینت را فریب دهند.

اگر کلاینت k پاسخ یکسان دریافت کند، f عدد از آن‌ها می‌تواند دروغ باشد. بنابراین، حداقل $k - f$ پاسخ از طرف سرورهای صادق آمده است. برای اینکه این پاسخ قابل اعتماد باشد، تعداد صادق‌ها باید بر تعداد خائن‌ها در این گروه غلبه کند:

$$k - f > f \implies k > 2f$$

کوچکترین عدد صحیحی که این شرط را برآورده می‌کند، $k = 2f + 1$ است. نتیجه بخش اول: یک کلاینت برای اطمینان کامل، باید حداقل $2f + 1$ پاسخ یکسان دریافت کند. این عدد تضمین می‌کند که آرای صادقانه $(f + 1)$ بر

آرای دروغین (f) برتری دارد.

بخش دوم: اثبات ریاضی (استخراج مستقیم فرمول)

اکنون با رویکردی مستقیم‌تر و ریاضی، فرمول را استخراج می‌کنیم. این رویکرد، همانطور که در متون جدید اشاره شد، دو مشکل اصلی ناشی از خطای بیزانسی را در نظر می‌گیرد:

مشکل اول: نودهایی که پیام ارسال نمی‌کنند.

در بدترین حالت، f نود خائن ممکن است تصمیم بگیرند هیچ پیامی ارسال نکنند. بنابراین، از n نود کل سیستم، ما تنها می‌توانیم روی دریافت پیام از $n - f$ نود حساب کنیم. این گروه، مجموعه اولیه پاسخ‌دهندگان ما را تشکیل می‌دهد.

مشکل دوم: نودهایی که پیام دروغین ارسال می‌کنند.

حالا باید بدترین سناریو را درون همین گروه $n - f$ نفری در نظر بگیریم. ممکن است f نود خائن، به جای سکوت، در میان این پاسخ‌دهندگان باشند و تظاهر به صداقت کنند.

برای یافتن تعداد پاسخ‌های واقعاً صادق در این گروه، باید این f نود خائن احتمالی را نیز از آن کم کنیم. تعداد پاسخ‌های تضمین‌شده صادق برابر است با:

$$(n - f) - f$$

شرط نهایی برای رسیدن به اجماع:

برای اینکه سیستم بتواند به یک تصمیم قابل اعتماد برسد، تعداد پاسخ‌های تضمین‌شده صادق باید اکیداً بیشتر از تعداد کل نودهای خائن (f) باشد. این شرط تضمین می‌کند که حتی اگر تمام خائن‌ها علیه یک تصمیم رأی دهند، باز هم آرای صادقانه پیروز می‌شود.

این شرط، ما را به نابرابری کلیدی زیر می‌رساند:

$$(n - f) - f > f$$

با ساده‌سازی این نابرابری:

$$n - 2f > f \implies n > 3f$$

از آنجایی که n باید یک عدد صحیح باشد، کوچکترین مقداری که در شرط $n > 3f$ صدق می‌کند، $n = 3f + 1$ است.

[۱] و [۲]

سوال ۲

بنظرم با توجه به توضیحات گفته شده این سناریو محتمل است.

دوره اول: شروع کار و اولین *crash*

در دوره اول، یک رهبر (مثلاً $S5$) انتخاب شده است و ورودی‌های $[1, 1]$ را ایجاد می‌کند. این ورودی‌ها با موفقیت به سرورهای $S1, S2, S4$ و $S5$ ارسال می‌شوند. سرور $S3$ فقط ورودی اول را دریافت می‌کند و سپس *crash* می‌شود. در این مرحله، وضعیت کاملاً معتبر و ممکن است.

دوره دوم: رهبری کوتاه و دومین *crash*

در دوره دوم، انتخابات جدیدی برگزار می‌شود و $S2$ به‌عنوان رهبر انتخاب می‌شود (زیرا لاگش از اکثریت به‌روزتر یا برابر است). به‌عنوان رهبر، $S2$ ورودی جدیدی از دوره خود در ایندکس ۳ اضافه می‌کند و لاگش به $[1, 1, 2]$ تبدیل می‌شود. نکته کلیدی من این است که $S2$ بلافاصله پس از تغییر لاگ داخلی و قبل از ارسال آن به دیگران، *crash* می‌شود. در این لحظه، لاگ‌ها به‌صورت زیر هستند:

$S2$: $[1, 1, 2]$ (آفلاین)

$S1, S4, S5$: $[1, 1]$

$S3$: $[1]$

دوره سوم: رهبری کوتاه‌تر و سومین *crash*

در دوره سوم، دوباره انتخابات برگزار می‌شود. $S1$ با لاگ $[1, 1]$ کاندیدا می‌شود و رای $S3, S4$ و $S5$ را می‌گیرد و رهبر می‌شود. $S1$ به‌عنوان رهبر، ورودی جدیدی از دوره خود در ایندکس ۳ اضافه می‌کند و لاگش به $[1, 1, 3]$ تبدیل می‌شود. نکته کلیدی دوم من این است که $S1$ نیز بلافاصله و قبل از ارسال لاگ جدیدش، *crash* می‌کند. در این لحظه، وضعیت بسیار ناهمگون و پیچیده شده است.

دوره چهارم: انتخابات نهایی و شروع همگرایی

وقتی $S2$ دوباره آنلاین می‌شود، برای رهبری در دوره چهارم کاندیدا می‌گردد. تحلیل رای‌گیری من کاملاً صحیح است: $S2$ با لاگ $[1, 1, 2]$ نسبت به $S3, S4$ و $S5$ که لاگ‌های قدیمی‌تری دارند، به‌روزتر است و رای اکثریت را کسب می‌کند و رهبر دوره چهارم می‌شود. اولین وظیفه $S2$ به‌عنوان رهبر جدید، همگام‌سازی لاگ دیگران با لاگ خودش است. او می‌بیند که لاگ $S4$ و $S5$ تا ایندکس ۲ با او یکی است، پس ورودی (ایندکس ۳، دوره ۲) را برایشان ارسال می‌کند و لاگ آن‌ها به $[1, 1, 2]$ آپدیت می‌شود. سپس $S2$ یک دستور جدید از کلاینت دریافت می‌کند و یک ورودی از دوره خود به لاگش اضافه می‌کند. لاگ نهایی $S2$ به $[1, 1, 2, 4]$ تبدیل می‌شود و هنوز فرصت نکرده آن را به بقیه بفرستد (مثلاً *crash* شده است).

(a)

به نظر من گزینه ۱ امکان‌پذیر نیست (مگر اینکه در الگوریتم *raft* این امکان باشد که $S3$ ، که لاگ قدیمی دارد، برای مدت کوتاهی دوباره راه‌اندازی شود و قبل از بروزرسانی لاگش، به $S1$ رای بدهد و مجدداً *crash* کند).
گزینه ۲ به نظرم غلط است با توجه به سناریویی که توضیح دادم، چون $S2$ لاگ بزرگ‌تری از $S1$ دارد و به او رای نمی‌دهد.

گزینه ۳ به نظرم امکان‌پذیر نیست چون لاگ $S3$ کامل‌ترین لاگ در ترم ۱ نیست.
گزینه ۴ به نظرم صحیح نیست چون $S3$ در حالت توقف (*stop*) است و نمی‌تواند رای دهد (مگر اینکه در الگوریتم *raft* این امکان باشد که $S3$ برای لحظه‌ای در انتخاب رهبر شرکت کند اما قبل از بروزرسانی لاگش به لاگ کامیت‌شده جدیدتر، *crash* کند؛ در آن صورت توجیه می‌شود که لاگش بدون بروزرسانی در رای‌گیری شرکت کرده).
گزینه ۵ نیز طبق سناریو امکان ندارد، چون اگر هر سه $S2$ ، $S4$ و $S5$ لاگ مربوط به ترم ۲ را دریافت کرده بودند، $S1$ در دوره بعد نمی‌توانست رهبر شود (زیرا لاگش قدیمی‌تر است و رای اکثریت را نمی‌توانست بگیرد)، در حالی که می‌دانیم $S1$ در دوره بعد رهبر شده است.

(b)

چون اکثریت لاگ $[1, 1, 2]$ را دارند، این لاگ‌ها کامیت نهایی شده‌اند. حال که رهبری در ترم چهارم به $S2$ رسیده است، ممکن است او لاگ مربوط به ترم ۴ را *Append* کند و اکثریت آن را بپذیرد و کامیت شود. بنابراین لاگ ترم ۴ هم می‌تواند در آینده به کامیت‌ها اضافه شود. تنها گزینه درست که این شرایط را رعایت کرده است، گزینه (c) یعنی $[1, 1, 2, 4]$ است (چون در ابتدا باید $[1, 1, 2]$ باشد که مطمئنیم توسط اکثریت پذیرفته و کامیت شده؛ این نکته فقط در گزینه (c) برقرار است). چون این لاگ کامیت‌شده است، در صورتی که سرورهای *crash* کرده به سیستم برگردند، لاگ آنها به این لاگ جدیدتر و بزرگ‌تر آپدیت می‌شود و همه آنها به همین لاگ خواهند رسید.
البته الان به ذهنم رسید که امکان دارد حالت گزینه a نیز رخ دهد، در صورتی برای ترم ۵، دوباره $s1$ رای بیاورد، می‌تواند لاگ‌ها را آپدیت کند. رای دهندگان به آن نیز چون لاگ آخر $s1$ مربوط به ترم ۳ است، و لاگ آخر خودشان برای ترم ۲ است، لاگ خودشان را به $[1, 1, 3]$ آپدیت می‌کنند. $s1$ می‌تواند ادامه دهد و لاگ مربوط به ترم ۵ را بگذارد. اما گزینه b بنظرم پذیرفته نیست، چون در صورتی که لاگ ترم ۴ توسط $s2$ که رهبر است، کامیت شده باشد، حتماً باید لاگ ترم ۲ نیز به همراه آن کامیت شده باشد و به همان گزینه c می‌رسیم. پس این گزینه غلط است.
[۳] و [۴]

سوال ۳

(a)

یک فرآیند پواسون (*Poisson Process*) برای مدل‌سازی مسائلی مناسب است که شامل شمارش تعداد دفعات وقوع یک رویداد خاص در یک بازه زمانی یا مکانی مشخص هستند. این فرآیند بر پایه چند فرض کلیدی بنا شده است:

۱. استقلال (*Independence*): وقوع یک رویداد بر احتمال وقوع رویداد دیگر تأثیری ندارد.
۲. نرخ میانگین ثابت (*Constant Average Rate*): نرخ میانگینی (λ) که رویدادها با آن رخ می‌دهند، در طول زمان ثابت است.

۳. عدم وقوع همزمان (*Non – simultaneity*): دو رویداد نمی‌توانند دقیقاً در یک لحظه رخ دهند.

۴. تناسب (*Proportionality*): احتمال وقوع یک رویداد در یک بازه بسیار کوچک، متناسب با طول آن بازه است. استخراج بلاک‌های بیت‌کوین به خوبی با یک فرآیند پواسون مدل‌سازی می‌شود، زیرا با این مفروضات هماهنگی دارد:
- استقلال: هر تلاش برای یافتن یک هش معتبر برای بلاک، یک آزمایش مستقل است. موفقیت یا شکست یک ماینر در یک تلاش، هیچ تأثیری بر نتیجه تلاش بعدی او یا تلاش‌های سایر ماینرها ندارد. این فرآیند دارای خاصیت بی‌حافظگی (*memoryless property*) است.

• نرخ میانگین ثابت: اگرچه نرخ هش کل شبکه نوسان می‌کند، پروتکل بیت‌کوین طوری طراحی شده است که میانگین زمان ایجاد بلاک را ثابت نگه دارد. هر ۲۰۱۶ بلاک، سختی استخراج (*mining difficulty*) به طور خودکار تنظیم می‌شود تا هدف یافتن یک بلاک جدید تقریباً هر ۱۰ دقیقه یک بار محقق شود. این مکانیزم تنظیم تضمین می‌کند که نرخ میانگین بلندمدت (λ) کشف بلاک، صرف‌نظر از تعداد ماینرهایی که به شبکه می‌پیوندند یا از آن خارج می‌شوند، ثابت باقی بماند.

- عدم وقوع همزمان: احتمال اینکه دو ماینر مختلف یک هش معتبر را دقیقاً در یک لحظه بی‌نهایت کوچک از زمان پیدا کنند، عملاً صفر است.

(b)

باید اثبات کنیم که اگر تعداد بلاک‌های استخراج شده از یک فرآیند پواسون پیروی کند، آنگاه فاصله زمانی بین بلاک‌های متوالی یک متغیر تصادفی مستقل و با توزیع یکسان (*i.i.d*) است که از توزیع نمایی پیروی می‌کند. اثبات:

فرض کنید $N(t)$ فرآیند پواسون نمایانگر تعداد بلاک‌های استخراج شده در بازه زمانی $[0, t]$ باشد. نرخ این فرآیند λ است که میانگین تعداد بلاک‌های استخراج شده در واحد زمان است. احتمال استخراج k بلاک در زمان t توسط تابع جرم احتمال پواسون داده می‌شود:

$$P(N(t) = k) = \frac{e^{-\lambda t} (\lambda t)^k}{k!} \quad (۱)$$

فرض کنید T_1 متغیر تصادفی نمایانگر زمان لازم برای استخراج اولین بلاک باشد. ما T_1 را پیدا کنیم. رویدادی که زمان لازم برای استخراج اولین بلاک بیشتر از مقدار t باشد (یعنی $T_1 > t$)، معادل رویدادی است که در بازه زمانی $[0, t]$ هیچ بلاکی استخراج نشده باشد (یعنی $N(t) = 0$). بنابراین، می‌توانیم بنویسیم:

$$P(T_1 > t) = P(N(t) = 0) \quad (۲)$$

با استفاده از فرمول پواسون برای $k = 0$

$$P(N(t) = 0) = \frac{(\lambda t)^0 e^{-\lambda t}}{0!} = e^{-\lambda t} \quad (۳)$$

این رابطه، تابع بقای (*survival function*) متغیر T_1 را به ما می‌دهد. تابع توزیع تجمعی (*CDF*)، $F_{T_1}(t)$ ، برابر با احتمال این است که زمان لازم برای استخراج اولین بلاک، کمتر یا مساوی t باشد:

$$F_{T_1}(t) = P(T_1 \leq t) = 1 - P(T_1 > t) = 1 - e^{-\lambda t}, \quad \text{for } t \geq 0 \quad (۴)$$

برای یافتن تابع چگالی احتمال (*PDF*)، $f_{T_1}(t)$ ، از تابع توزیع تجمعی نسبت به t مشتق می‌گیریم:

$$\frac{d}{dt} F_{T_1}(t) = \frac{d}{dt} (1 - e^{-\lambda t}) = \lambda e^{-\lambda t} \quad (۵)$$

این تابع چگالی احتمال، $f_{T_1}(t) = \lambda e^{-\lambda t}$ ، تعریف دقیق توزیع نمایی (*exponential distribution*) با پارامتر نرخ λ است.

به دلیل خاصیت بی‌حافظگی فرآیند پواسون، نرخ کشف بلاک مستقل از رویدادهای گذشته است. این بدان معناست که زمان از بلاک اول تا دوم، T_2

، از همان منطق و توزیع T_1 پیروی می‌کند. بنابراین، تمام فواصل زمانی بین بلاک‌ها ($T_1, T_2, \dots$) مستقل و با توزیع یکسان (*i.i.d*) به صورت $Exponential(\lambda)$ هستند.

(c)

این یک مسئله کلاسیک از رقابت توزیع‌های نمایی است.

فرض کنید T_A

زمان لازم برای پیدا کردن بلاک بعدی توسط ماینر A و T_B

زمان لازم برای پیدا کردن آن توسط ماینر B باشد. از آنجایی که استخراج بلاک از فرآیند پواسون پیروی می‌کند،

زمان لازم برای پیدا کردن بلاک توسط هر ماینر از توزیع نمایی پیروی می‌کند.

نرخ‌ها برابر با نرخ هش (قدرت پردازش) هر ماینر است:

$$T_A \sim \text{Exponential}(\lambda_A) \bullet$$

$$T_B \sim \text{Exponential}(\lambda_B) \bullet$$

توابع چگالی احتمال آن‌ها به صورت زیر است:

$$f_A(t) = \lambda_A e^{-\lambda_A t} \bullet$$

$$f_B(t) = \lambda_B e^{-\lambda_B t} \bullet$$

ما می‌خواهیم احتمال اینکه ماینر A بلاک را زودتر از ماینر B پیدا کند، یعنی $P(T_A < T_B)$ ، را محاسبه کنیم.

می‌توانیم این مسئله را با شرطی کردن بر روی زمان T_B و انتگرال‌گیری روی تمام مقادیر ممکن T_B حل کنیم:

$$P(T_A < T_B) = \int_0^\infty P(T_A < T_B \mid T_B = t_B) f_B(t_B) dt_B$$

با فرض اینکه $T_B = t_B$ ، احتمال $P(T_A < t_B)$ برابر با تابع توزیع تجمعی T_A در نقطه t_B است که برابر است با $(1 - e^{-\lambda_A t_B})$.

$$\begin{aligned} P(T_A < T_B) &= \int_0^\infty (1 - e^{-\lambda_A t_B}) (\lambda_B e^{-\lambda_B t_B}) dt_B \\ &= \int_0^\infty \lambda_B e^{-\lambda_B t_B} dt_B - \int_0^\infty \lambda_B e^{-(\lambda_A + \lambda_B) t_B} dt_B \end{aligned}$$

انتگرال اول، کل احتمال توزیع نمایی برای B است که برابر با ۱ می‌شود. برای انتگرال دوم:

$$\begin{aligned}
&= 1 - \lambda_B \left[-\frac{1}{\lambda_A + \lambda_B} e^{-(\lambda_A + \lambda_B)t_B} \right]_0^\infty \\
&= 1 - \lambda_B \left(0 - \left(-\frac{1}{\lambda_A + \lambda_B} \right) \right) \\
&= 1 - \frac{\lambda_B}{\lambda_A + \lambda_B} \\
&= \frac{(\lambda_A + \lambda_B) - \lambda_B}{\lambda_A + \lambda_B} \\
&= \frac{\lambda_A}{\lambda_A + \lambda_B}
\end{aligned}$$

بنابراین، احتمال اینکه ماینر A بلاک را قبل از ماینر B پیدا کند برابر است با:

$$P(T_A < T_B) = \frac{\lambda_A}{\lambda_A + \lambda_B}$$

به طور شهودی، احتمال اینکه یک ماینر زودتر از دیگری موفق شود، برابر با نسبت نرخ هش او به مجموع نرخ هش دو ماینر رقیب است. قدرت پردازش کل شبکه، λ_T ، در رقابت بین A و B بی‌تأثیر است.

سوال ۴

متغیر تصادفی X از توزیع هندسی با احتمال موفقیت p پیروی می‌کند. تابع جرم احتمال آن به صورت $P(X = k) = (1 - p)^{k-1}p$ برای $k = 1, 2, \dots$ است.

امید ریاضی و واریانس آن به شرح زیر است:

$$E(X) = \mu = \frac{1}{p} \bullet$$

$$Var(X) = \sigma^2 = \frac{1-p}{p^2} \bullet$$

ما به دنبال یافتن یک کران بالا برای احتمال $P(X \geq cE(X))$ هستیم که می‌توان آن را به صورت $P(X \geq \frac{c}{p})$ برای ثابت $c > 1$ نوشت.

۱. نامساوی مارکوف (*Markov's Inequality*)

برای هر متغیر تصادفی غیرمنفی X و هر ثابت $a > 0$ ، نامساوی مارکوف بیان می‌کند:

$$P(X \geq a) \leq \frac{E(X)}{a}$$

از آنجایی که توزیع هندسی برای $k \geq 1$ تعریف می‌شود، X یک متغیر تصادفی غیرمنفی است. ما می‌توانیم این نامساوی را با $a = cE(X)$ به کار ببریم.

$$P(X \geq cE(X)) \leq \frac{E(X)}{cE(X)}$$

ساده‌سازی این عبارت، کران زیر را به دست می‌دهد:

$$P(X \geq cE(X)) \leq \frac{1}{c}$$

۲. نامساوی چبیشف (Chebyshev's Inequality)

برای یک متغیر تصادفی X با میانگین متناهی μ و واریانس متناهی σ^2 ، نامساوی چبیشف برای هر ثابت $k > 0$ بیان می‌کند:

$$P(|X - \mu| \geq k) \leq \frac{\sigma^2}{k^2}$$

رویداد $X \geq c\mu$ معادل است با $X - \mu \geq (c - 1)\mu$ چون $c > 1$ ، مقدار $(c - 1)\mu$ مثبت است. رویداد $X - \mu \geq (c - 1)\mu$ زیرمجموعه‌ای از رویداد $|X - \mu| \geq (c - 1)\mu$ است. بنابراین:

$$P(X \geq c\mu) = P(X - \mu \geq (c - 1)\mu) \leq P(|X - \mu| \geq (c - 1)\mu)$$

می‌توانیم نامساوی چبیشف را با $k = (c - 1)\mu = (c - 1)\frac{1}{p}$ به کار ببریم.

$$P(X \geq cE(X)) \leq \frac{\text{Var}(X)}{((c - 1)E(X))^2}$$

با جایگذاری عبارات میانگین و واریانس:

$$P(X \geq cE(X)) \leq \frac{\frac{1-p}{p^2}}{\left(\frac{c-1}{p}\right)^2} = \frac{\frac{1-p}{p^2}}{\frac{(c-1)^2}{p^2}}$$

این عبارت به کران زیر ساده می‌شود:

$$P(X \geq cE(X)) \leq \frac{1-p}{(c-1)^2}$$

۳. نامساوی چرنوف (*Chernoff's Inequality*)

ما می‌خواهیم احتمال $P(X \geq a)$ را کران دار کنیم.

به ازای هر عدد مثبت $t > 0$ ، تابع e^{tx} اکیداً صعودی است. این بدان معناست که رویداد $X \geq a$ با رویداد $e^{tX} \geq e^{ta}$ یکسان است. بنابراین:

$$P(X \geq a) = P(e^{tX} \geq e^{ta})$$

اکنون، متغیر تصادفی جدید $Y = e^{tX}$ را در نظر بگیرید. از آنجایی که $t > 0$ و تابع نمایی همیشه مثبت است، Y یک متغیر تصادفی غیرمنفی است. اکنون می‌توانیم نامساوی مارکوف را بر روی Y اعمال کنیم. نامساوی مارکوف بیان می‌کند که برای یک متغیر تصادفی غیرمنفی Y ، داریم:

$$P(Y \geq b) \leq \frac{E[Y]}{b}$$

با اعمال این نامساوی بر روی عبارت ما، با فرض $Y = e^{tX}$ و $b = e^{ta}$ ، به دست می‌آوریم:

$$P(e^{tX} \geq e^{ta}) \leq \frac{E[e^{tX}]}{e^{ta}}$$

با بازنویسی سمت راست، به کران میانی زیر می‌رسیم:

$$P(X \geq a) \leq e^{-ta} E[e^{tX}]$$

توجه داشته باشید که عبارت $E[e^{tX}]$ تعریف تابع مولد گشتاور (*Moment Generating Function – MGF*) برای متغیر تصادفی X است که اغلب به صورت $M_X(t)$ نوشته می‌شود.

نامساوی‌ای که در گام قبل به دست آوردیم برای هر مقدار $t > 0$ برقرار است. از آنجایی که ما به دنبال محکم‌ترین (کوچک‌ترین) کران بالای ممکن هستیم، باید مقداری از t را پیدا کنیم که عبارت سمت راست را کمینه (*minimize*) کند.

این مرحله بهینه‌سازی همان چیزی است که عملگر $\min$ را به فرمول اضافه می‌کند و در نتیجه، نهایی‌ترین و قوی‌ترین شکل کران به دست می‌آید: پس کران چرنوف با استفاده از تابع مولد گشتاور (*MGF*) استخراج می‌شود. برای یک متغیر تصادفی X و هر $a > 0$ ، کران به صورت زیر است:

$$P(X \geq a) \leq \min_{t>0} e^{-ta} E[e^{tX}]$$

در اینجا $a = cE(X) = \frac{c}{p}$ است. تابع مولد گشتاور یک متغیر تصادفی با توزیع هندسی برابر است با:

$$M_X(t) = E[e^{tX}] = \frac{pe^t}{1 - (1-p)e^t}, \quad \text{for } t < -\ln(1-p)$$

کران به صورت $P\left(X \geq \frac{c}{p}\right) \leq e^{-tc/p} \frac{pe^t}{1-(1-p)e^t}$ است. برای یافتن بهترین کران، باید این عبارت را نسبت به t کمینه کنیم. اگر عبارت را $f(t)$ بنامیم، با صفر قرار دادن مشتق $\ln(f(t))$ کمینه را می‌یابیم.

$$\frac{d}{dt} \ln(f(t)) = \frac{d}{dt} \left(-\frac{tc}{p} + \ln(pe^t) - \ln(1 - (1-p)e^t) \right) = 0$$

حل معادله برای e^t مقدار بهینه را نتیجه می‌دهد:

$$e^t = \frac{c-p}{c(1-p)}$$

این مقدار برای t معتبر است زیرا شرط $t < -\ln(1-p)$ را (به دلیل $c > 1$) ارضا می‌کند. با جایگذاری این مقدار بهینه e^t در فرمول کران $M_X(t) e^{-\frac{tc}{p}}$ داریم:

$$P\left(X \geq \frac{c}{p}\right) \leq \left(\frac{c-p}{c(1-p)}\right)^{-c/p} \left(\frac{pe^t}{1-(1-p)e^t}\right)$$

پس از جایگذاری و ساده‌سازی، کران چرنوف به صورت زیر است:

$$P(X \geq cE(X)) \leq \left(\frac{c-p}{c(1-p)}\right)^{-c/p} \left(\frac{c-p}{1-p}\right)$$

[۵]

مراجع

- [1] “The math behind pbft,” <https://medium.com/codechain/why-n-3f-1-in-the-byzantine-fault-tolerance-system-c3ca6bab8fe9>, 2018.
- [2] “Why is byzantine fault tolerance 3f+1,” <https://www.linkedin.com/pulse/why-byzantine-fault-tolerance-3f1-gerald-lee>, 2023.
- [3] “What is raft ,” <https://raft.github.io/>, 2016.
- [4] “The raft consensus algorithm,” <https://www.youtube.com/watch?v=vYp4LYbnnW8>, 2016.
- [5] “Geometric Distribution Bounds,” <https://g.co/gemini/share/Geometric-Dist-Bounds>, 2025.